Java Interview Preparation

Part 3: Top 100 Core Java Algorithmic & Coding Questions

Section 1: Arrays, Strings & Basic Data Structures (Q1–Q20)

Q1: Reverse an array of integers in-place without using extra space.

```
// Time: O(N), Space: O(1)
public static void reverseArray(int[] arr) {
    if (arr == null || arr.length <= 1) return;
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;
        left++;
        right--;
    }
}
```

Q2: Find the duplicate number in an array of N+1 integers containing values from 1 to N (Floyd's Cycle Detection).

```
// Time: O(N), Space: O(1)
public static int findDuplicate(int[] nums) {
    int slow = nums[0];
    int fast = nums[0];
    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);

    slow = nums[0];
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}
```

Q3: Two Sum — Find indices of two numbers in an array that add up to a target sum.

```
// Time: O(N), Space: O(N)
public static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement)) {
            return new int[]{map.get(complement), i};
        }
        map.put(nums[i], i);
    }
    return new int[]{};
}
```

Q4: Check if two strings are valid Anagrams.

```
// Time: O(N), Space: O(1) assuming 26 English lowercase letters
public static boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] counts = new int[26];
    for (int i = 0; i < s.length(); i++) {
        counts[s.charAt(i) - 'a']++;
        counts[t.charAt(i) - 'a']--;
    }
    for (int count : counts) {
        if (count != 0) return false;
    }
    return true;
}
```

Q5: Check if a given string is a valid Palindrome ignoring non-alphanumeric characters and case.

```
// Time: O(N), Space: O(1)
public static boolean isPalindrome(String s) {
    if (s == null) return false;
    int i = 0, j = s.length() - 1;
    while (i < j) {
        while (i < j && !Character.isLetterOrDigit(s.charAt(i))) i++;
        while (i < j && !Character.isLetterOrDigit(s.charAt(j))) j--;
        if (Character.toLowerCase(s.charAt(i)) != Character.toLowerCase(s.charAt(j))) {
            return false;
        }
        i++;
        j--;
    }
    return true;
}
```

Q6: Find the majority element in an array (Boyer-Moore Voting Algorithm).

```
// Time: O(N), Space: O(1)
public static int majorityElement(int[] nums) {
    int candidate = 0, count = 0;
    for (int num : nums) {
        if (count == 0) {
            candidate = num;
        }
        count += (num == candidate) ? 1 : -1;
    }
    return candidate;
}
```

Q7: Move all zeros in an integer array to the end while preserving the relative order of non-zero elements.

```
// Time: O(N), Space: O(1)
public static void moveZeroes(int[] nums) {
    int lastNonZeroFoundAt = 0;
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] != 0) {
            int temp = nums[lastNonZeroFoundAt];
            nums[lastNonZeroFoundAt++] = nums[i];
            nums[i] = temp;
        }
    }
}
```

Q8: Rotate an array to the right by K steps in-place.

```
// Time: O(N), Space: O(1)
public static void rotate(int[] nums, int k) {
    k %= nums.length;
    reverse(nums, 0, nums.length - 1);
    reverse(nums, 0, k - 1);
    reverse(nums, k, nums.length - 1);
}

private static void reverse(int[] nums, int start, int end) {
    while (start < end) {
        int temp = nums[start];
        nums[start++] = nums[end];
        nums[end--] = temp;
    }
}
```

Q9: Find the missing number in an array containing N distinct numbers in range [0,N].

```
// Time: O(N), Space: O(1)
public static int missingNumber(int[] nums) {
    int n = nums.length;
    int expectedSum = n * (n + 1) / 2;
    int actualSum = 0;
    for (int num : nums) actualSum += num;
    return expectedSum - actualSum;
}
```

Q10: Find the longest common prefix across an array of strings.

```
// Time: O(S) where S is sum of all characters, Space: O(1)
public static String longestCommonPrefix(String[] strs) {
    if (strs == null || strs.length == 0) return "";
    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
        while (strs[i].indexOf(prefix) != 0) {
            prefix = prefix.substring(0, prefix.length() - 1);
            if (prefix.isEmpty()) return "";
        }
    }
    return prefix;
}
```

Q11: Find the first unique (non-repeating) character in a string and return its index.

```
// Time: O(N), Space: O(1)
public static int firstUniqChar(String s) {
    int[] count = new int[26];
    for (int i = 0; i < s.length(); i++) count[s.charAt(i) - 'a']++;
    for (int i = 0; i < s.length(); i++) {
        if (count[s.charAt(i) - 'a'] == 1) return i;
    }
    return -1;
}
```

Q12: Reverse individual words in a string while preserving word order.

```
// Time: O(N), Space: O(N)
public static String reverseWords(String s) {
    String[] words = s.trim().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = words.length - 1; i >= 0; i--) {
        sb.append(words[i]);
        if (i > 0) sb.append(" ");
    }
    return sb.toString();
}
```

Q13: Compress a character array in-place using consecutive repeating character counts (String Compression).

```
// Time: O(N), Space: O(1)
public static int compress(char[] chars) {
    int indexAns = 0, index = 0;
    while (index < chars.length) {
        char currentChar = chars[index];
        int count = 0;
        while (index < chars.length && chars[index] == currentChar) {
            index++;
            count++;
        }
        chars[indexAns++] = currentChar;
        if (count > 1) {
            for (char c : String.valueOf(count).toCharArray()) {
                chars[indexAns++] = c;
            }
        }
    }
    return indexAns;
}
```

Q14: Find the single number in a non-empty array where every element appears twice except for one (Bit XOR).

```
// Time: O(N), Space: O(1)
public static int singleNumber(int[] nums) {
    int result = 0;
    for (int num : nums) {
        result ^= num; // XOR cancels out duplicate numbers
    }
    return result;
}
```

Q15: Check if a string contains valid balanced parentheses (), {}, [].

```
// Time: O(N), Space: O(N)
public static boolean isValidParentheses(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push(')');
        else if (c == '{') stack.push('}');
        else if (c == '[') stack.push(']');
        else if (stack.isEmpty() || stack.pop() != c) return false;
    }
    return stack.isEmpty();
}
```

Q16: Remove element in-place and return the new array length.

```
// Time: O(N), Space: O(1)
public static int removeElement(int[] nums, int val) {
    int k = 0;
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] != val) {
            nums[k++] = nums[i];
        }
    }
    return k;
}
```

Q17: Merge two sorted arrays nums1 and nums2 into nums1 in sorted order.

```
// Time: O(M + N), Space: O(1)
public static void merge(int[] nums1, int m, int[] nums2, int n) {
    int p1 = m - 1, p2 = n - 1, p = m + n - 1;
    while (p1 >= 0 && p2 >= 0) {
        if (nums1[p1] > nums2[p2]) {
            nums1[p--] = nums1[p1--];
        } else {
            nums1[p--] = nums2[p2--];
        }
    }
    while (p2 >= 0) {
        nums1[p--] = nums2[p2--];
    }
}
```

Q18: Find maximum profit from a single buy/sell stock transaction (Best Time to Buy and Sell Stock).

```
// Time: O(N), Space: O(1)
public static int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE;
    int maxProfit = 0;
    for (int price : prices) {
        if (price < minPrice) {
            minPrice = price;
        } else if (price - minPrice > maxProfit) {
            maxProfit = price - minPrice;
        }
    }
    return maxProfit;
}
```

Q19: Compute Product of Array Except Self without using division operation.

```
// Time: O(N), Space: O(1) auxiliary (excluding output array)
public static int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    res[0] = 1;
    for (int i = 1; i < n; i++) {
        res[i] = res[i - 1] * nums[i - 1];
    }
    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        res[i] *= right;
        right *= nums[i];
    }
    return res;
}
```

Q20: Convert a Roman Numeral string to its integer representation.

```
// Time: O(N), Space: O(1)
public static int romanToInt(String s) {
    Map<Character, Integer> map = Map.of(
        'I', 1, 'V', 5, 'X', 10, 'L', 50, 'C', 100, 'D', 500, 'M', 1000
    );
    int total = 0, prev = 0;
    for (int i = s.length() - 1; i >= 0; i--) {
        int curr = map.get(s.charAt(i));
        if (curr < prev) total -= curr;
        else total += curr;
        prev = curr;
    }
    return total;
}
```

Section 2: Two Pointers, Sliding Window & Binary Search (Q21–Q40)

Q21: Find length of the Longest Substring Without Repeating Characters (Sliding Window).

```
// Time: O(N), Space: O(N)
public static int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> map = new HashMap<>();
    int maxLen = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (map.containsKey(c)) {
            left = Math.max(left, map.get(c) + 1);
        }
        map.put(c, right);
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Q22: Container With Most Water — Find two lines that together with x-axis form a container holding maximum water.

```
// Time: O(N), Space: O(1)
public static int maxArea(int[] height) {
    int left = 0, right = height.length - 1;
    int maxWater = 0;
    while (left < right) {
        int h = Math.min(height[left], height[right]);
        maxWater = Math.max(maxWater, h * (right - left));
        if (height[left] < height[right]) left++;
        else right--;
    }
    return maxWater;
}
```

Q23: 3Sum — Find all unique triplets in an array that sum up to zero.

```
// Time: O(N^2), Space: O(1) (excluding result space)
public static List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        int left = i + 1, right = nums.length - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                res.add(List.of(nums[i], nums[left], nums[right]));
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
                left++;
                right--;
            } else if (sum < 0) left++;
            else right--;
        }
    }
    return res;
}
```

Q24: Minimum Size Subarray Sum — Find minimal length of a contiguous subarray whose sum is >= target.

```
// Time: O(N), Space: O(1)
public static int minSubArrayLen(int target, int[] nums) {
    int minLen = Integer.MAX_VALUE;
    int left = 0, currentSum = 0;
    for (int right = 0; right < nums.length; right++) {
        currentSum += nums[right];
        while (currentSum >= target) {
            minLen = Math.min(minLen, right - left + 1);
            currentSum -= nums[left++];
        }
    }
    return minLen == Integer.MAX_VALUE ? 0 : minLen;
}
```

Q25: Implement Binary Search on a sorted array.

```
// Time: O(log N), Space: O(1)
public static int binarySearch(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // Avoids integer overflow
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Q26: Search in Rotated Sorted Array.

```
// Time: O(log N), Space: O(1)
public static int searchRotated(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        // Left half sorted
        if (nums[left] <= nums[mid]) {
            if (nums[left] <= target && target < nums[mid]) right = mid - 1;
            else left = mid + 1;
        } else { // Right half sorted
            if (nums[mid] < target && target <= nums[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
    return -1;
}
```

Q27: Find Minimum in Rotated Sorted Array.

```
// Time: O(log N), Space: O(1)
public static int findMinRotated(int[] nums) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] > nums[right]) left = mid + 1;
        else right = mid;
    }
    return nums[left];
}
```

Q28: Find First and Last Position of Element in Sorted Array.

```
// Time: O(log N), Space: O(1)
public static int[] searchRange(int[] nums, int target) {
    int first = findBound(nums, target, true);
    if (first == -1) return new int[]{-1, -1};
    int last = findBound(nums, target, false);
    return new int[]{first, last};
}

private static int findBound(int[] nums, int target, boolean isFirst) {
    int left = 0, right = nums.length - 1, bound = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            bound = mid;
            if (isFirst) right = mid - 1;
            else left = mid + 1;
        } else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return bound;
}
```

Q29: Trapping Rain Water — Calculate total trapped water after raining.

```
// Time: O(N), Space: O(1)
public static int trap(int[] height) {
    int left = 0, right = height.length - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (left < right) {
        if (height[left] < height[right]) {
            if (height[left] >= leftMax) leftMax = height[left];
            else water += leftMax - height[left];
            left++;
        } else {
            if (height[right] >= rightMax) rightMax = height[right];
            else water += rightMax - height[right];
            right--;
        }
    }
    return water;
}
```

Q30: Find the Peak Element in an array (where element is strictly greater than neighbors).

```
// Time: O(log N), Space: O(1)
public static int findPeakElement(int[] nums) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] > nums[mid + 1]) right = mid;
        else left = mid + 1;
    }
    return left;
}
```

Q31: Find Maximum Consecutive Ones in a binary array.

```
// Time: O(N), Space: O(1)
public static int findMaxConsecutiveOnes(int[] nums) {
    int maxCount = 0, currentCount = 0;
    for (int num : nums) {
        if (num == 1) {
            currentCount++;
            maxCount = Math.max(maxCount, currentCount);
        } else {
            currentCount = 0;
        }
    }
    return maxCount;
}
```

Q32: Longest Repeating Character Replacement — Find longest substring length containing same letter after substituting at most K characters.

```
// Time: O(N), Space: O(1)
public static int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int maxCount = 0, maxLength = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        maxCount = Math.max(maxCount, ++count[s.charAt(right) - 'A']);
        while (right - left + 1 - maxCount > k) {
            count[s.charAt(left) - 'A']--;
            left++;
        }
        maxLength = Math.max(maxLength, right - left + 1);
    }
    return maxLength;
}
```

Q33: Find All Anagrams in a String (Sliding Window).

```
// Time: O(N), Space: O(1)
public static List<Integer> findAnagrams(String s, String p) {
    List<Integer> result = new ArrayList<>();
    if (s.length() < p.length()) return result;
    int[] pCount = new int[26], sCount = new int[26];
    for (int i = 0; i < p.length(); i++) {
        pCount[p.charAt(i) - 'a']++;
        sCount[s.charAt(i) - 'a']++;
    }
    if (Arrays.equals(pCount, sCount)) result.add(0);
    for (int i = p.length(); i < s.length(); i++) {
        sCount[s.charAt(i) - 'a']++;
        sCount[s.charAt(i - p.length()) - 'a']--;
        if (Arrays.equals(pCount, sCount)) {
            result.add(i - p.length() + 1);
        }
    }
    return result;
}
```

Q34: Kth Smallest Element in a Sorted Matrix.

```
// Time: O(N log(Max - Min)), Space: O(1)
public static int kthSmallest(int[][] matrix, int k) {
    int n = matrix.length;
    int low = matrix[0][0], high = matrix[n - 1][n - 1];
    while (low < high) {
        int mid = low + (high - low) / 2;
        int count = countLessEqual(matrix, mid);
        if (count < k) low = mid + 1;
        else high = mid;
    }
    return low;
}

private static int countLessEqual(int[][] matrix, int mid) {
    int count = 0, col = matrix.length - 1;
    for (int row = 0; row < matrix.length; row++) {
        while (col >= 0 && matrix[row][col] > mid) col--;
        count += (col + 1);
    }
    return count;
}
```

Q35: Intersection of Two Arrays II (containing duplicate elements).

```
// Time: O(N + M), Space: O(Min(N, M))
public static int[] intersect(int[] nums1, int[] nums2) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int num : nums1) map.put(num, map.getOrDefault(num, 0) + 1);
    List<Integer> result = new ArrayList<>();
    for (int num : nums2) {
        if (map.getOrDefault(num, 0) > 0) {
            result.add(num);
            map.put(num, map.get(num) - 1);
        }
    }
    return result.stream().mapToInt(i -> i).toArray();
}
```

Q36: Compute Square Root of X rounded down to nearest integer using Binary Search.

```
// Time: O(log X), Space: O(1)
public static int mySqrt(int x) {
    if (x == 0 || x == 1) return x;
    int left = 1, right = x / 2, ans = 0;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (mid <= x / mid) {
            ans = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return ans;
}
```

Q37: Search a 2D Matrix (where rows are sorted and first integer of row > last of previous row).

```
// Time: O(log(M * N)), Space: O(1)
public static boolean searchMatrix(int[][] matrix, int target) {
    int m = matrix.length, n = matrix[0].length;
    int left = 0, right = m * n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        int val = matrix[mid / n][mid % n];
        if (val == target) return true;
        if (val < target) left = mid + 1;
        else right = mid - 1;
    }
    return false;
}
```

Q38: Find Peak Index in a Mountain Array.

```
// Time: O(log N), Space: O(1)
public static int peakIndexInMountainArray(int[] arr) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] < arr[mid + 1]) left = mid + 1;
        else right = mid;
    }
    return left;
}
```

Q39: Subarray Sum Equals K — Count total continuous subarrays whose sum equals K.

```
// Time: O(N), Space: O(N)
public static int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixSumMap = new HashMap<>();
    prefixSumMap.put(0, 1);
    int count = 0, currentSum = 0;
    for (int num : nums) {
        currentSum += num;
        if (prefixSumMap.containsKey(currentSum - k)) {
            count += prefixSumMap.get(currentSum - k);
        }
        prefixSumMap.put(currentSum, prefixSumMap.getOrDefault(currentSum, 0) + 1);
    }
    return count;
}
```

Q40: Longest Consecutive Sequence — Find length of longest consecutive elements sequence in an unsorted array in O(N) time.

```
// Time: O(N), Space: O(N)
public static int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int num : nums) set.add(num);
    int maxLen = 0;
    for (int num : set) {
        // Only start sequence from the lowest number in a range
        if (!set.contains(num - 1)) {
            int currentNum = num;
            int currentLen = 1;
            while (set.contains(currentNum + 1)) {
                currentNum++;
                currentLen++;
            }
            maxLen = Math.max(maxLen, currentLen);
        }
    }
    return maxLen;
}
```

Section 3: Linked Lists, Stacks & Queues (Q41–Q60)

Q41: Reverse a Singly Linked List iteratively and recursively.

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}

// Iterative — Time: O(N), Space: O(1)
public static ListNode reverseListIterative(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode nextTemp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}

// Recursive — Time: O(N), Space: O(N) call stack
public static ListNode reverseListRecursive(ListNode head) {
    if (head == null || head.next == null) return head;
    ListNode p = reverseListRecursive(head.next);
    head.next.next = head;
    head.next = null;
    return p;
}
```

Q42: Detect if a Linked List contains a Cycle (Floyd's Tortoise and Hare).

```
// Time: O(N), Space: O(1)
public static boolean hasCycle(ListNode head) {
    if (head == null) return false;
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

Q43: Merge Two Sorted Linked Lists into a single sorted list.

```
// Time: O(N + M), Space: O(1)
public static ListNode mergeTwoLists(ListNode list1, ListNode list2) {
    ListNode dummy = new ListNode(-1);
    ListNode curr = dummy;
    while (list1 != null && list2 != null) {
        if (list1.val <= list2.val) {
            curr.next = list1;
            list1 = list1.next;
        } else {
            curr.next = list2;
            list2 = list2.next;
        }
        curr = curr.next;
    }
    curr.next = (list1 != null) ? list1 : list2;
    return dummy.next;
}
```

Q44: Remove Nth Node From End of Linked List in one pass.

```
// Time: O(N), Space: O(1)
public static ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode first = dummy, second = dummy;
    for (int i = 0; i <= n; i++) {
        first = first.next;
    }
    while (first != null) {
        first = first.next;
        second = second.next;
    }
    second.next = second.next.next;
    return dummy.next;
}
```

Q45: Find Middle of the Linked List.

```
// Time: O(N), Space: O(1)
public static ListNode middleNode(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Q46: Check if a Linked List is a Palindrome.

```
// Time: O(N), Space: O(1)
public static boolean isPalindrome(ListNode head) {
    if (head == null || head.next == null) return true;
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    ListNode prev = null, curr = slow;
    while (curr != null) {
        ListNode nextTemp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = nextTemp;
    }
    ListNode p1 = head, p2 = prev;
    while (p2 != null) {
        if (p1.val != p2.val) return false;
        p1 = p1.next;
        p2 = p2.next;
    }
    return true;
}
```

Q47: Find Intersection Point of Two Linked Lists.

```
// Time: O(N + M), Space: O(1)
public static ListNode getIntersectionNode(ListNode headA, ListNode headB) {
    if (headA == null || headB == null) return null;
    ListNode pA = headA, pB = headB;
    while (pA != pB) {
        pA = (pA == null) ? headB : pA.next;
        pB = (pB == null) ? headA : pB.next;
    }
    return pA;
}
```

Q48: Reverse Linked List in K-Group.

```
// Time: O(N), Space: O(1)
public static ListNode reverseKGroup(ListNode head, int k) {
    ListNode countNode = head;
    int count = 0;
    while (countNode != null && count < k) {
        countNode = countNode.next;
        count++;
    }
    if (count == k) {
        ListNode curr = head, prev = null, next = null;
        for (int i = 0; i < k; i++) {
            next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        head.next = reverseKGroup(curr, k);
        return prev;
    }
    return head;
}
```

Q49: Copy List with Random Pointer (Deep Copy).

```
class Node {
    int val;
    Node next, random;
    Node(int val) { this.val = val; }
}

// Time: O(N), Space: O(N)
public static Node copyRandomList(Node head) {
    if (head == null) return null;
    Map<Node, Node> map = new HashMap<>();
    Node curr = head;
    while (curr != null) {
        map.put(curr, new Node(curr.val));
        curr = curr.next;
    }
    curr = head;
    while (curr != null) {
        map.get(curr).next = map.get(curr.next);
        map.get(curr).random = map.get(curr.random);
        curr = curr.next;
    }
    return map.get(head);
}
```

Q50: Implement a Min Stack supporting push, pop, top, and getMin in O(1) time.

```
class MinStack {
    private Deque<Integer> stack = new ArrayDeque<>();
    private Deque<Integer> minStack = new ArrayDeque<>();

    public void push(int val) {
        stack.push(val);
        if (minStack.isEmpty() || val <= minStack.peek()) {
            minStack.push(val);
        }
    }

    public void pop() {
        if (stack.peek().equals(minStack.peek())) {
            minStack.pop();
        }
        stack.pop();
    }

    public int top() { return stack.peek(); }
    public int getMin() { return minStack.peek(); }
}
```

Q51: Implement Queue using Stacks.

```
class MyQueue {
    private Deque<Integer> in = new ArrayDeque<>();
    private Deque<Integer> out = new ArrayDeque<>();

    public void push(int x) { in.push(x); }

    public int pop() {
        peek();
        return out.pop();
    }

    public int peek() {
        if (out.isEmpty()) {
            while (!in.isEmpty()) out.push(in.pop());
        }
        return out.peek();
    }

    public boolean empty() { return in.isEmpty() && out.isEmpty(); }
}
```

Q52: Evaluate Reverse Polish Notation (Postfix Expression).

```
// Time: O(N), Space: O(N)
public static int evalRPN(String[] tokens) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (String token : tokens) {
        if ("+-*/".contains(token)) {
            int b = stack.pop();
            int a = stack.pop();
            switch (token) {
                case "+": stack.push(a + b); break;
                case "-": stack.push(a - b); break;
                case "*": stack.push(a * b); break;
                case "/": stack.push(a / b); break;
            }
        } else {
            stack.push(Integer.parseInt(token));
        }
    }
    return stack.pop();
}
```

Q53: Daily Temperatures — Find number of days to wait until a warmer temperature (Monotonic Stack).

```
// Time: O(N), Space: O(N)
public static int[] dailyTemperatures(int[] temperatures) {
    int n = temperatures.length;
    int[] answer = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()]) {
            int prevIndex = stack.pop();
            answer[prevIndex] = i - prevIndex;
        }
        stack.push(i);
    }
    return answer;
}
```

Q54: Largest Rectangle in Histogram (Monotonic Stack).

```
// Time: O(N), Space: O(N)
public static int largestRectangleArea(int[] heights) {
    int n = heights.length, maxArea = 0;
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i];
        while (!stack.isEmpty() && h < heights[stack.peek()]) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}
```

Q55: Sliding Window Maximum using Monotonic Deque.

```
// Time: O(N), Space: O(K)
public static int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || k <= 0) return new int[0];
    int n = nums.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) deque.pollFirst();
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) deque.pollLast();
        deque.offerLast(i);
        if (i >= k - 1) result[i - k + 1] = nums[deque.peekFirst()];
    }
    return result;
}
```

Q56: LRU Cache implementation using LinkedHashMap or custom doubly linked list + hash map.

```
class LRUCache {
    private final int capacity;
    private final Map<Integer, Integer> map;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.map = new LinkedHashMap<>(capacity, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
                return size() > capacity;
            }
        };
    }

    public int get(int key) { return map.getOrDefault(key, -1); }
    public void put(int key, int value) { map.put(key, value); }
}
```

Q57: Add Two Numbers represented as reversed Linked Lists.

```
// Time: O(Max(N, M)), Space: O(Max(N, M))
public static ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;
    int carry = 0;
    while (l1 != null || l2 != null || carry != 0) {
        int sum = carry;
        if (l1 != null) { sum += l1.val; l1 = l1.next; }
        if (l2 != null) { sum += l2.val; l2 = l2.next; }
        carry = sum / 10;
        curr.next = new ListNode(sum % 10);
        curr = curr.next;
    }
    return dummy.next;
}
```

Q58: Reorder List — Given L0 -> L1 -> … -> Ln, reorder to L0 -> Ln -> L1 -> Ln-1 …

```
// Time: O(N), Space: O(1)
public static void reorderList(ListNode head) {
    if (head == null || head.next == null) return;
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    ListNode prev = null, curr = slow.next;
    slow.next = null;
    while (curr != null) {
        ListNode tmp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = tmp;
    }
    ListNode first = head, second = prev;
    while (second != null) {
        ListNode tmp1 = first.next, tmp2 = second.next;
        first.next = second;
        second.next = tmp1;
        first = tmp1;
        second = tmp2;
    }
}
```

Q59: Sort Linked List in O(N log N) time using Merge Sort.

```
// Time: O(N log N), Space: O(log N) stack
public static ListNode sortList(ListNode head) {
    if (head == null || head.next == null) return head;
    ListNode prev = null, slow = head, fast = head;
    while (fast != null && fast.next != null) {
        prev = slow;
        slow = slow.next;
        fast = fast.next.next;
    }
    prev.next = null;
    ListNode l1 = sortList(head);
    ListNode l2 = sortList(slow);
    return mergeTwoLists(l1, l2);
}
```

Q60: Flatten a Multilevel Doubly Linked List.

```
class NodeMulti {
    int val;
    NodeMulti prev, next, child;
}

// Time: O(N), Space: O(N) stack
public static NodeMulti flatten(NodeMulti head) {
    if (head == null) return null;
    NodeMulti curr = head;
    while (curr != null) {
        if (curr.child != null) {
            NodeMulti next = curr.next;
            NodeMulti childFlat = flatten(curr.child);
            curr.next = childFlat;
            childFlat.prev = curr;
            curr.child = null;
            while (curr.next != null) curr = curr.next;
            if (next != null) {
                curr.next = next;
                next.prev = curr;
            }
        }
        curr = curr.next;
    }
    return head;
}
```

Section 4: Trees, Graphs & Recursion (Q61–Q80)

Q61: Binary Tree Inorder Traversal (Iterative & Recursive).

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

// Iterative — Time: O(N), Space: O(N)
public static List<Integer> inorderTraversal(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;
    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }
        curr = stack.pop();
        res.add(curr.val);
        curr = curr.right;
    }
    return res;
}
```

Q62: Binary Tree Level Order Traversal (BFS).

```
// Time: O(N), Space: O(N)
public static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        res.add(level);
    }
    return res;
}
```

Q63: Maximum Depth of Binary Tree.

```
// Time: O(N), Space: O(H) where H is height
public static int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

Q64: Invert/Flip a Binary Tree.

```
// Time: O(N), Space: O(H)
public static TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode left = invertTree(root.left);
    TreeNode right = invertTree(root.right);
    root.left = right;
    root.right = left;
    return root;
}
```

Q65: Check if Binary Tree is Symmetric (Mirror image of itself).

```
// Time: O(N), Space: O(H)
public static boolean isSymmetric(TreeNode root) {
    return root == null || isMirror(root.left, root.right);
}

private static boolean isMirror(TreeNode t1, TreeNode t2) {
    if (t1 == null && t2 == null) return true;
    if (t1 == null || t2 == null) return false;
    return (t1.val == t2.val) && isMirror(t1.left, t2.right) && isMirror(t1.right, t2.left);
}
```

Q66: Validate Binary Search Tree (BST).

```
// Time: O(N), Space: O(H)
public static boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}

private static boolean validate(TreeNode node, Integer low, Integer high) {
    if (node == null) return true;
    if ((low != null && node.val <= low) || (high != null && node.val >= high)) return false;
    return validate(node.left, low, node.val) && validate(node.right, node.val, high);
}
```

Q67: Lowest Common Ancestor (LCA) in Binary Search Tree.

```
// Time: O(H), Space: O(1)
public static TreeNode lowestCommonAncestorBST(TreeNode root, TreeNode p, TreeNode q) {
    TreeNode curr = root;
    while (curr != null) {
        if (p.val < curr.val && q.val < curr.val) curr = curr.left;
        else if (p.val > curr.val && q.val > curr.val) curr = curr.right;
        else return curr;
    }
    return null;
}
```

Q68: Lowest Common Ancestor (LCA) in General Binary Tree.

```
// Time: O(N), Space: O(H)
public static TreeNode lowestCommonAncestorBT(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode left = lowestCommonAncestorBT(root.left, p, q);
    TreeNode right = lowestCommonAncestorBT(root.right, p, q);
    if (left != null && right != null) return root;
    return (left != null) ? left : right;
}
```

Q69: Binary Tree Maximum Path Sum.

```
// Time: O(N), Space: O(H)
private static int maxSum = Integer.MIN_VALUE;

public static int maxPathSum(TreeNode root) {
    maxGain(root);
    return maxSum;
}

private static int maxGain(TreeNode node) {
    if (node == null) return 0;
    int leftGain = Math.max(maxGain(node.left), 0);
    int rightGain = Math.max(maxGain(node.right), 0);
    int currentPathSum = node.val + leftGain + rightGain;
    maxSum = Math.max(maxSum, currentPathSum);
    return node.val + Math.max(leftGain, rightGain);
}
```

Q70: Serialize and Deserialize a Binary Tree.

```
public class Codec {
    // Time: O(N), Space: O(N)
    public String serialize(TreeNode root) {
        if (root == null) return "null,";
        return root.val + "," + serialize(root.left) + serialize(root.right);
    }

    public TreeNode deserialize(String data) {
        Queue<String> nodes = new LinkedList<>(Arrays.asList(data.split(",")));
        return buildTree(nodes);
    }

    private TreeNode buildTree(Queue<String> nodes) {
        String val = nodes.poll();
        if ("null".equals(val)) return null;
        TreeNode node = new TreeNode(Integer.parseInt(val));
        node.left = buildTree(nodes);
        node.right = buildTree(nodes);
        return node;
    }
}
```

Q71: Kth Smallest Element in a BST.

```
// Time: O(H + K), Space: O(H)
public static int kthSmallest(TreeNode root, int k) {
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;
    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }
        curr = stack.pop();
        if (--k == 0) return curr.val;
        curr = curr.right;
    }
    return -1;
}
```

Q72: Construct Binary Tree from Preorder and Inorder Traversal.

```
// Time: O(N), Space: O(N)
private static int preorderIndex;
private static Map<Integer, Integer> inorderMap;

public static TreeNode buildTree(int[] preorder, int[] inorder) {
    preorderIndex = 0;
    inorderMap = new HashMap<>();
    for (int i = 0; i < inorder.length; i++) inorderMap.put(inorder[i], i);
    return arrayToTree(preorder, 0, inorder.length - 1);
}

private static TreeNode arrayToTree(int[] preorder, int left, int right) {
    if (left > right) return null;
    int rootVal = preorder[preorderIndex++];
    TreeNode root = new TreeNode(rootVal);
    root.left = arrayToTree(preorder, left, inorderMap.get(rootVal) - 1);
    root.right = arrayToTree(preorder, inorderMap.get(rootVal) + 1, right);
    return root;
}
```

Q73: Number of Islands (DFS/BFS grid search).

```
// Time: O(M * N), Space: O(M * N)
public static int numIslands(char[][] grid) {
    if (grid == null || grid.length == 0) return 0;
    int count = 0;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == '1') {
                count++;
                dfsIsland(grid, r, c);
            }
        }
    }
    return count;
}

private static void dfsIsland(char[][] grid, int r, int c) {
    if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length || grid[r][c] == '0') return;
    grid[r][c] = '0'; // Sink island element
    dfsIsland(grid, r + 1, c);
    dfsIsland(grid, r - 1, c);
    dfsIsland(grid, r, c + 1);
    dfsIsland(grid, r, c - 1);
}
```

Q74: Clone an Undirected Graph.

```
class NodeGraph {
    int val;
    List<NodeGraph> neighbors = new ArrayList<>();
    NodeGraph(int val) { this.val = val; }
}

// Time: O(V + E), Space: O(V)
public static NodeGraph cloneGraph(NodeGraph node) {
    if (node == null) return null;
    Map<NodeGraph, NodeGraph> visited = new HashMap<>();
    return dfsClone(node, visited);
}

private static NodeGraph dfsClone(NodeGraph node, Map<NodeGraph, NodeGraph> visited) {
    if (visited.containsKey(node)) return visited.get(node);
    NodeGraph clone = new NodeGraph(node.val);
    visited.put(node, clone);
    for (NodeGraph neighbor : node.neighbors) {
        clone.neighbors.add(dfsClone(neighbor, visited));
    }
    return clone;
}
```

Q75: Course Schedule — Topological Sort / Cycle Detection in Directed Graph (Kahn's Algorithm BFS).

```
// Time: O(V + E), Space: O(V + E)
public static boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adj = new ArrayList<>();
    int[] indegree = new int[numCourses];
    for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
    for (int[] pre : prerequisites) {
        adj.get(pre[1]).add(pre[0]);
        indegree[pre[0]]++;
    }
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) queue.offer(i);
    }
    int visitedCount = 0;
    while (!queue.isEmpty()) {
        int course = queue.poll();
        visitedCount++;
        for (int neighbor : adj.get(course)) {
            if (--indegree[neighbor] == 0) queue.offer(neighbor);
        }
    }
    return visitedCount == numCourses;
}
```

Q76: Rotting Oranges — Multi-Source BFS on 2D Matrix.

```
// Time: O(M * N), Space: O(M * N)
public static int orangesRotting(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    Queue<int[]> queue = new LinkedList<>();
    int freshCount = 0;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 2) queue.offer(new int[]{r, c});
            else if (grid[r][c] == 1) freshCount++;
        }
    }
    if (freshCount == 0) return 0;
    int minutes = 0;
    int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
    while (!queue.isEmpty()) {
        int size = queue.size();
        boolean rottedNew = false;
        for (int i = 0; i < size; i++) {
            int[] curr = queue.poll();
            for (int[] d : dirs) {
                int nr = curr[0] + d[0], nc = curr[1] + d[1];
                if (nr >= 0 && nc >= 0 && nr < rows && nc < cols && grid[nr][nc] == 1) {
                    grid[nr][nc] = 2;
                    queue.offer(new int[]{nr, nc});
                    freshCount--;
                    rottedNew = true;
                }
            }
        }
        if (rottedNew) minutes++;
    }
    return freshCount == 0 ? minutes : -1;
}
```

Q77: Word Ladder — Shortest transformation sequence using BFS.

```
// Time: O(M^2 * N), Space: O(M * N)
public static int ladderLength(String beginWord, String endWord, List<String> wordList) {
    Set<String> wordSet = new HashSet<>(wordList);
    if (!wordSet.contains(endWord)) return 0;
    Queue<String> queue = new LinkedList<>();
    queue.offer(beginWord);
    int level = 1;
    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            String curr = queue.poll();
            char[] chars = curr.toCharArray();
            for (int j = 0; j < chars.length; j++) {
                char orig = chars[j];
                for (char c = 'a'; c <= 'z'; c++) {
                    if (c == orig) continue;
                    chars[j] = c;
                    String next = new String(chars);
                    if (next.equals(endWord)) return level + 1;
                    if (wordSet.contains(next)) {
                        queue.offer(next);
                        wordSet.remove(next);
                    }
                }
                chars[j] = orig;
            }
        }
        level++;
    }
    return 0;
}
```

Q78: Graph Valid Tree — Check if undirected graph forms a single valid tree (Disjoint Set Union / Union-Find).

```
// Time: O(V + E * alpha(V)), Space: O(V)
public static boolean validTree(int n, int[][] edges) {
    if (edges.length != n - 1) return false;
    int[] parent = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
    for (int[] edge : edges) {
        int root1 = find(parent, edge[0]);
        int root2 = find(parent, edge[1]);
        if (root1 == root2) return false; // Cycle detected
        parent[root1] = root2;
    }
    return true;
}

private static int find(int[] parent, int i) {
    if (parent[i] == i) return i;
    return parent[i] = find(parent, parent[i]);
}
```

Q79: Dijkstra's Algorithm — Single-source shortest paths in a weighted graph with non-negative edge weights.

```
// Time: O((V + E) log V), Space: O(V + E)
public static int[] dijkstra(int n, List<List<int[]>> adj, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int u = curr[0], d = curr[1];
        if (d > dist[u]) continue;
        for (int[] edge : adj.get(u)) {
            int v = edge[0], weight = edge[1];
            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.offer(new int[]{v, dist[v]});
            }
        }
    }
    return dist;
}
```

Q80: Flatten Binary Tree to Linked List in-place.

```
// Time: O(N), Space: O(1)
public static void flattenTree(TreeNode root) {
    TreeNode curr = root;
    while (curr != null) {
        if (curr.left != null) {
            TreeNode rightmost = curr.left;
            while (rightmost.right != null) rightmost = rightmost.right;
            rightmost.right = curr.right;
            curr.right = curr.left;
            curr.left = null;
        }
        curr = curr.right;
    }
}
```

Section 5: Dynamic Programming, Backtracking & Sorting (Q81–Q100)

Q81: Climbing Stairs — N steps, can take 1 or 2 steps at a time (Fibonacci DP).

```
// Time: O(N), Space: O(1)
public static int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

Q82: Coin Change — Minimum coins needed to make up an amount.

```
// Time: O(Amount * N), Space: O(Amount)
public static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i) {
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Q83: Longest Increasing Subsequence (LIS) in O(N log N) time.

```
// Time: O(N log N), Space: O(N)
public static int lengthOfLIS(int[] nums) {
    int[] tails = new int[nums.length];
    int len = 0;
    for (int x : nums) {
        int i = 0, j = len;
        while (i < j) {
            int m = (i + j) / 2;
            if (tails[m] < x) i = m + 1;
            else j = m;
        }
        tails[i] = x;
        if (i == len) len++;
    }
    return len;
}
```

Q84: 0/1 Knapsack Problem — Maximum value that can fit into a given weight capacity W.

```
// Time: O(N * W), Space: O(W)
public static int knapsack(int[] weights, int[] values, int W) {
    int n = weights.length;
    int[] dp = new int[W + 1];
    for (int i = 0; i < n; i++) {
        for (int w = W; w >= weights[i]; w--) {
            dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }
    return dp[W];
}
```

Q85: Longest Common Subsequence (LCS) between two strings.

```
// Time: O(M * N), Space: O(M * N)
public static int longestCommonSubsequence(String text1, String text2) {
    int m = text1.length(), n = text2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[m][n];
}
```

Q86: Edit Distance (Levenshtein Distance) — Minimum operations (insert, delete, replace) to convert string A to B.

```
// Time: O(M * N), Space: O(M * N)
public static int minDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = 1 + Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp[i][j - 1]));
            }
        }
    }
    return dp[m][n];
}
```

Q87: Maximum Subarray Sum (Kadane's Algorithm).

```
// Time: O(N), Space: O(1)
public static int maxSubArray(int[] nums) {
    int maxSoFar = nums[0];
    int currentMax = nums[0];
    for (int i = 1; i < nums.length; i++) {
        currentMax = Math.max(nums[i], currentMax + nums[i]);
        maxSoFar = Math.max(maxSoFar, currentMax);
    }
    return maxSoFar;
}
```

Q88: House Robber — Maximum money robbed without alerting security (no adjacent houses).

```
// Time: O(N), Space: O(1)
public static int rob(int[] nums) {
    int prevMax = 0, currMax = 0;
    for (int num : nums) {
        int temp = currMax;
        currMax = Math.max(prevMax + num, currMax);
        prevMax = temp;
    }
    return currMax;
}
```

Q89: Word Break — Check if a string can be segmented into space-separated dictionary words.

```
// Time: O(N^2 * M), Space: O(N)
public static boolean wordBreak(String s, List<String> wordDict) {
    Set<String> wordSet = new HashSet<>(wordDict);
    boolean[] dp = new boolean[s.length() + 1];
    dp[0] = true;
    for (int i = 1; i <= s.length(); i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && wordSet.contains(s.substring(j, i))) {
                dp[i] = true;
                break;
            }
        }
    }
    return dp[s.length()];
}
```

Q90: Unique Paths in a Grid (M x N) moving only right or down.

```
// Time: O(M * N), Space: O(N)
public static int uniquePaths(int m, int n) {
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] += dp[j - 1];
        }
    }
    return dp[n - 1];
}
```

Q91: Subsets — Generate all possible subsets (Power Set) via Backtracking.

```
// Time: O(N * 2^N), Space: O(N)
public static List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrackSubsets(0, nums, new ArrayList<>(), result);
    return result;
}

private static void backtrackSubsets(int start, int[] nums, List<Integer> current, List<List<Int
    result.add(new ArrayList<>(current));
    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);
        backtrackSubsets(i + 1, nums, current, result);
        current.remove(current.size() - 1);
    }
}
```

Q92: Permutations — Generate all unique permutations of an array.

```
// Time: O(N * N!), Space: O(N)
public static List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrackPermute(nums, new ArrayList<>(), new boolean[nums.length], result);
    return result;
}

private static void backtrackPermute(int[] nums, List<Integer> current, boolean[] used, List<Lis
    if (current.size() == nums.length) {
        result.add(new ArrayList<>(current));
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;
        used[i] = true;
        current.add(nums[i]);
        backtrackPermute(nums, current, used, result);
        current.remove(current.size() - 1);
        used[i] = false;
    }
}
```

Q93: Combination Sum — Find all combinations summing up to target (elements reusable).

```
// Time: O(2^T), Space: O(T) where T is target / min element
public static List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    backtrackCombo(0, candidates, target, new ArrayList<>(), result);
    return result;
}

private static void backtrackCombo(int start, int[] candidates, int remain, List<Integer> curren
    if (remain == 0) {
        result.add(new ArrayList<>(current));
        return;
    }
    for (int i = start; i < candidates.length; i++) {
        if (candidates[i] <= remain) {
            current.add(candidates[i]);
            backtrackCombo(i, candidates, remain - candidates[i], current, result);
            current.remove(current.size() - 1);
        }
    }
}
```

Q94: N-Queens Problem — Place N non-attacking queens on an N x N chessboard.

```
// Time: O(N!), Space: O(N)
public static List<List<String>> solveNQueens(int n) {
    List<List<String>> result = new ArrayList<>();
    char[][] board = new char[n][n];
    for (char[] row : board) Arrays.fill(row, '.');
    backtrackQueens(0, board, new boolean[n], new boolean[2 * n], new boolean[2 * n], result);
    return result;
}

private static void backtrackQueens(int r, char[][] board, boolean[] cols, boolean[] d1, boolean
    int n = board.length;
    if (r == n) {
        List<String> list = new ArrayList<>();
        for (char[] row : board) list.add(new String(row));
        result.add(list);
        return;
    }
    for (int c = 0; c < n; c++) {
        int id1 = r - c + n, id2 = r + c;
        if (cols[c] || d1[id1] || d2[id2]) continue;
        board[r][c] = 'Q';
        cols[c] = d1[id1] = d2[id2] = true;
        backtrackQueens(r + 1, board, cols, d1, d2, result);
        board[r][c] = '.';
        cols[c] = d1[id1] = d2[id2] = false;
    }
}
```

Q95: Quick Sort implementation with in-place partitioning.

```
// Time: O(N log N) average, O(N^2) worst; Space: O(log N)
public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

private static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
        }
    }
    int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
    return i + 1;
}
```

Q96: Merge Sort implementation.

```
// Time: O(N log N), Space: O(N)
public static void mergeSort(int[] arr, int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

private static void merge(int[] arr, int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int[] L = new int[n1], R = new int[n2];
    System.arraycopy(arr, l, L, 0, n1);
    System.arraycopy(arr, m + 1, R, 0, n2);
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

Q97: Kth Largest Element in an Array using QuickSelect in O(N) average time.

```
// Time: O(N) average, Space: O(1)
public static int findKthLargest(int[] nums, int k) {
    int target = nums.length - k;
    int low = 0, high = nums.length - 1;
    while (low <= high) {
        int p = partition(nums, low, high);
        if (p == target) return nums[p];
        else if (p < target) low = p + 1;
        else high = p - 1;
    }
    return -1;
}
```

Q98: Merge Intervals — Combine overlapping intervals.

```
// Time: O(N log N), Space: O(N)
public static int[][] mergeIntervals(int[][] intervals) {
    if (intervals.length <= 1) return intervals;
    Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));
    List<int[]> result = new ArrayList<>();
    int[] current = intervals[0];
    result.add(current);
    for (int[] interval : intervals) {
        if (interval[0] <= current[1]) {
            current[1] = Math.max(current[1], interval[1]);
        } else {
            current = interval;
            result.add(current);
        }
    }
    return result.toArray(new int[0][]);
}
```

Q99: Task Scheduler — Minimum CPU time slots needed to execute all tasks given a cooling period N.

```
// Time: O(Tasks), Space: O(1)
public static int leastInterval(char[] tasks, int n) {
    int[] freqs = new int[26];
    for (char t : tasks) freqs[t - 'A']++;
    Arrays.sort(freqs);
    int maxFreq = freqs[25];
    int idleSlots = (maxFreq - 1) * n;
    for (int i = 24; i >= 0 && freqs[i] > 0; i--) {
        idleSlots -= Math.min(freqs[i], maxFreq - 1);
    }
    return idleSlots > 0 ? tasks.length + idleSlots : tasks.length;
}
```

Q100: Find All Duplicate Subtrees in a Binary Tree using Postorder Serialization + Hash Mapping.